Backtracking: Subset Sum

Design and Analysis of Algorithms

Dhannya SM

Exhaustive Search

- In a number of combinatorial problems, we must find one (or more) solution(s) with a special property from among an exponentially large number of solutions.
- In principle, such a solution can be found by performing an exhaustive search in the pool of solutions.
- In practical, this may not be a possible option due to the staggeringly large number of solutions.

Backtracking

- A more intelligent variation of the exhaustive search
- Construct solutions one component at a time
- If a partially constructed solution can be developed further without violating any constraints, then take the first remaining option for the next component
- If there are no legitimate options for the next component, no further move is made in that direction
- Then, the algorithm *backtracks* to replace the last component of the partially constructed solution with its next option.

Backtracking

- A *state-space tree* is used to represent this kind of processing.
- Its root represents an initial state before the search begins
- The nodes in Level 1 represents the choices made for the first component of a solution, that in Level 2 for the second component and so on.
- A promising node is one that may lead to a complete solution.
- Complete solutions are arrived at the leaves.

Subset-Sum

- *Problem*: Given a list $A = \{a_1, a_2, \ldots, a_n\}$ of positive integers and an integer $d > 0$, find a subset $A' \subseteq A$ such that the sum of elements in A' is equal to d.
  For ex: if $A = \{1, 5, 8\}$ and $d = 13$, then $A' = \{5, 8\}$ is a solution.
- Note that some of the instances of this problem may not have any solution at all.
  For ex: if $A = \{1, 5, 8\}$ and $d = 12$, then no solution exists.

Backtracking Algorithm for Subset Sum

Let S be a set of elements and d is the expected sum of subsets.

a) Start with an empty solution set S'.

b) Add the next (first element in the first step) element from the list to S'.

c) If the elements in S' adds to d, then print S' as the output.

d) If S' is not feasible or if we have reached the end of the set, then backtrack to the previous component (recursively) until we find the most suitable value.

e) If the subset is feasible, then repeat step 2.

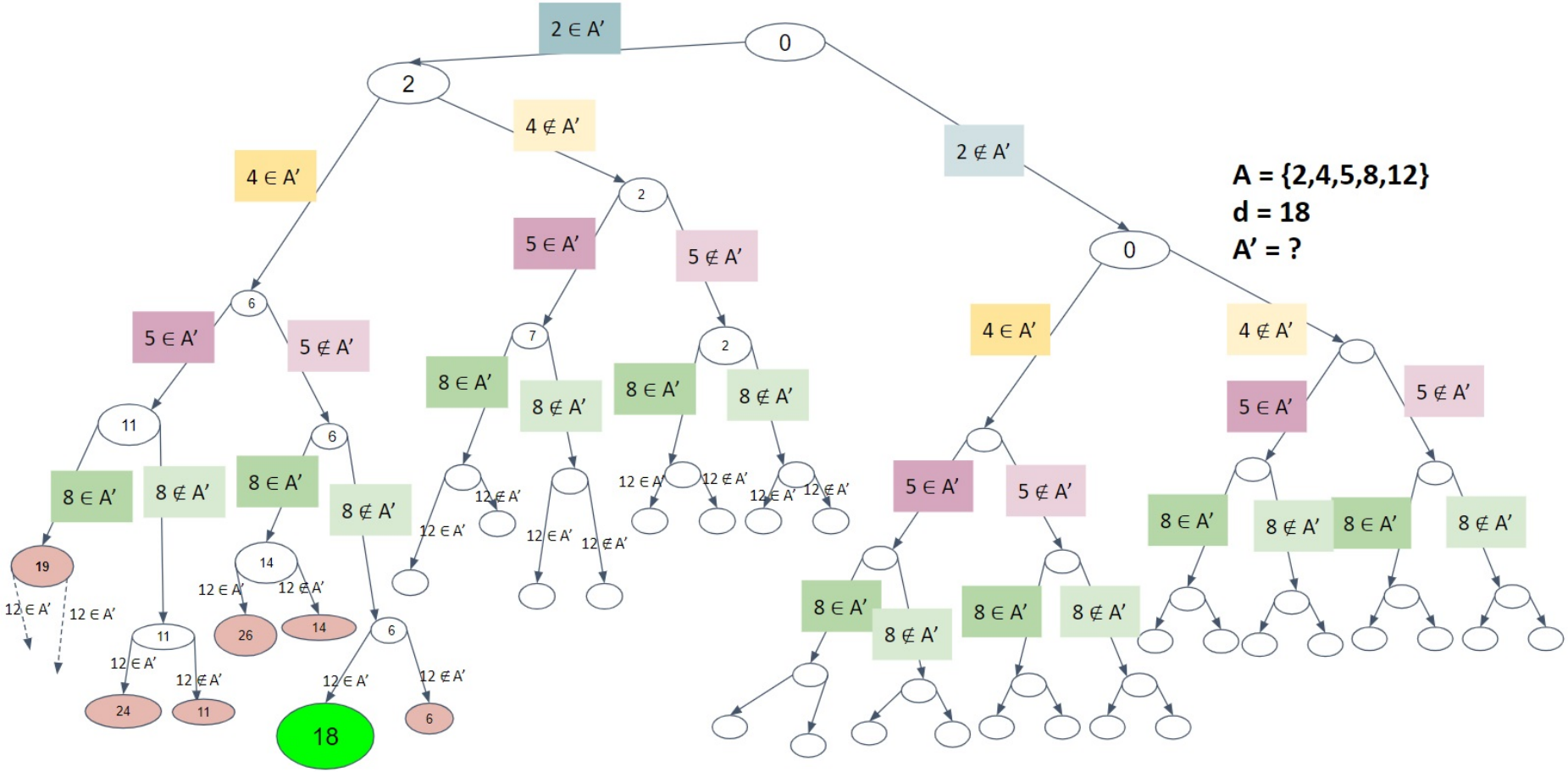
f) If we have reached the end of the set without finding a suitable subset and if no backtracking is possible then print "No solution".

State-space tree for Subset-Sum

- Let $A = \{2, 4, 5, 8, 12\}$ and $d = 18$

State-space tree for Subset-Sum

- Let $A = \{2, 4, 5, 8, 12\}$ and $d = 18$
- The state-space tree for this will look like this:



Time complexity

- Can be as bad as exponential.
- In this example, it will be $O(2^n)$.
- If the number of branches in the state-space tree is k, then the complexity will be $O(k^n)$.

Summary

- The technique of backtracking is based on exhaustive search.
- Non-promising searches are discontinued, and the algorithm goes back to the previous branch to take another path of the state-space tree.
- No guarantee in reducing the complexity from exponential, but the number of searches may be reduced drastically for a reasonable number of inputs.